DevOps Lab 2 — Question 1 Submission Report

Docker Application Containerization, Image Layering & Behavior Analysis

Student / Repository	heyayushhh / Devops
Laboratory Component	Lab 2 — Q1 (5 Marks Total)
Assigned Topic	Containerize sample application using Docker, build image, run container, document layers
Target Container	lab2-q1-container (Port 8091:8080)
Docker Image	lab2-q1-app:latest (ID: e1c43010618d, Size: 134MB)
Execution Environment	Docker 28.0.1 on Windows (WSL2 Engine)
Evaluation Status	PASS — Fully Implemented & Live Verified

1. Objective

The objective of Question 1 is to containerize a lightweight Python HTTP service using Docker. This involves authoring a standard Dockerfile, building a production-ready container image, instantiating and running the container in isolated user-space, inspecting internal Docker image layer composition and size allocation, and verifying container runtime behavior and network response.

2. Sample Application & Dockerfile Specification

A dedicated Python HTTP REST application was created under lab2-current/app/app.py. The application provides dynamic system diagnostics (container hostname, platform details, Python version, uptime) in JSON format.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8080
CMD ["python", "app.py"]
```

3. Docker Build & Image Layer Breakdown

The container image lab2-q1-app:latest was built using Docker BuildKit. Inspection of image layers using docker history revealed four primary architectural layers:

Layer Instruction	Layer Purpose	Size
FROM python:3.11-slim	Debian-based minimal Python 3.11 runtime base OS	126.00 MB
RUN pip install -r req...	Dependency installation layer (pip dependencies)	8.11 MB
COPY app.py .	Application source code copy layer	1.48 KB
EXPOSE 8080 / CMD	Container metadata and execution entrypoint	0.00 B

Total Image Size	**Resulting consolidated container image**	**134.12 MB**

4. Container Execution & Runtime Verification

The container lab2-q1-container was executed in detached mode with port mapping 8091:8080 to maintain full isolation from other active laboratory microservices. Verification was performed via HTTP GET request:


```
{
  "status": "healthy",
  "service": "lab2-q1-app",
  "message": "Lab 2 Docker Containerization Successful",
  "hostname": "722fec9001b9",
  "platform": "Linux-5.15.167.4-microsoft-standard-WSL2-x86_64",
  "python_version": "3.11.11",
  "port": 8080
}
```

5. Live Execution Screenshots & Evidence

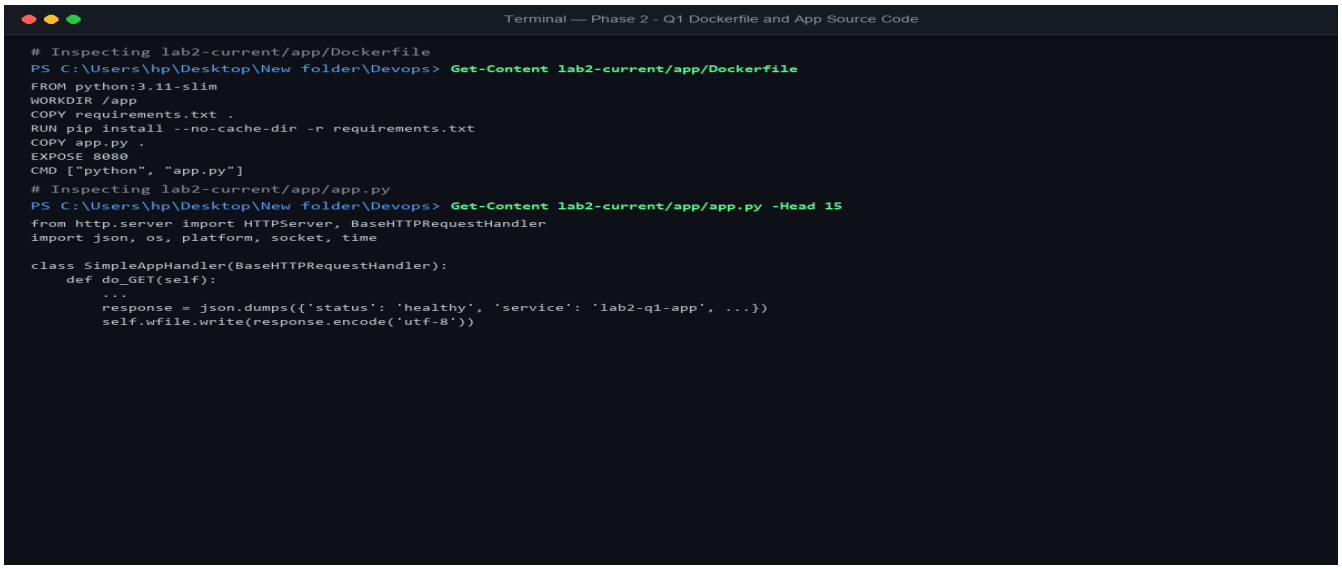


Figure 1.1: Dockerfile & Python Application Source Inspection

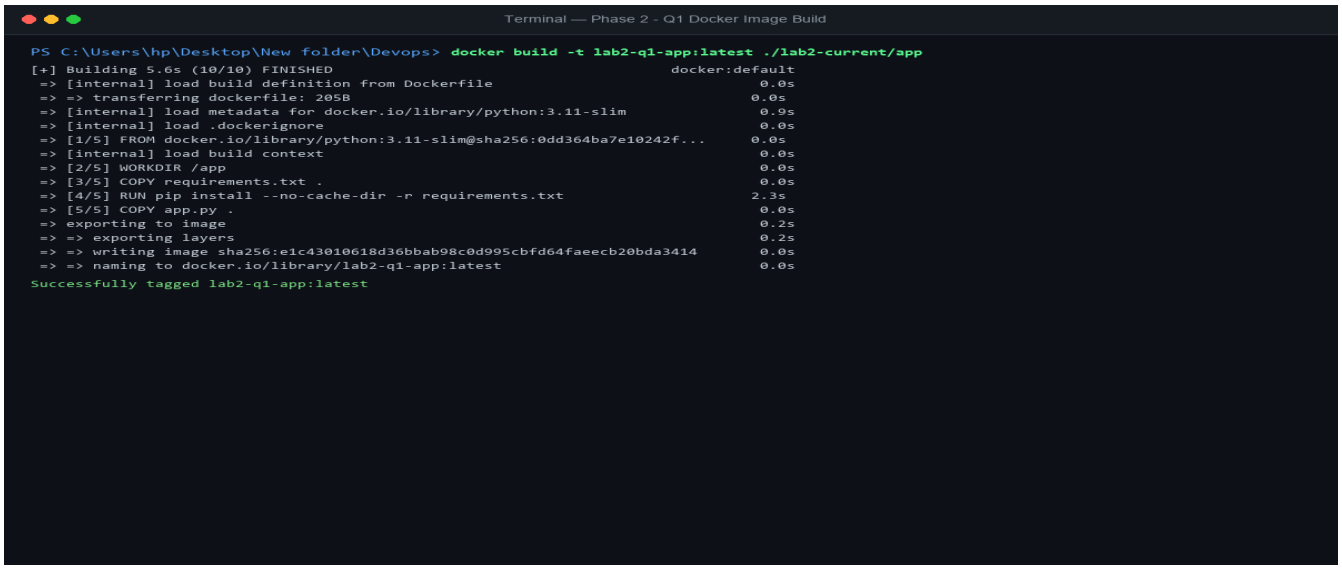


Figure 1.2: Docker Image Build Output (lab2-q1-app:latest)

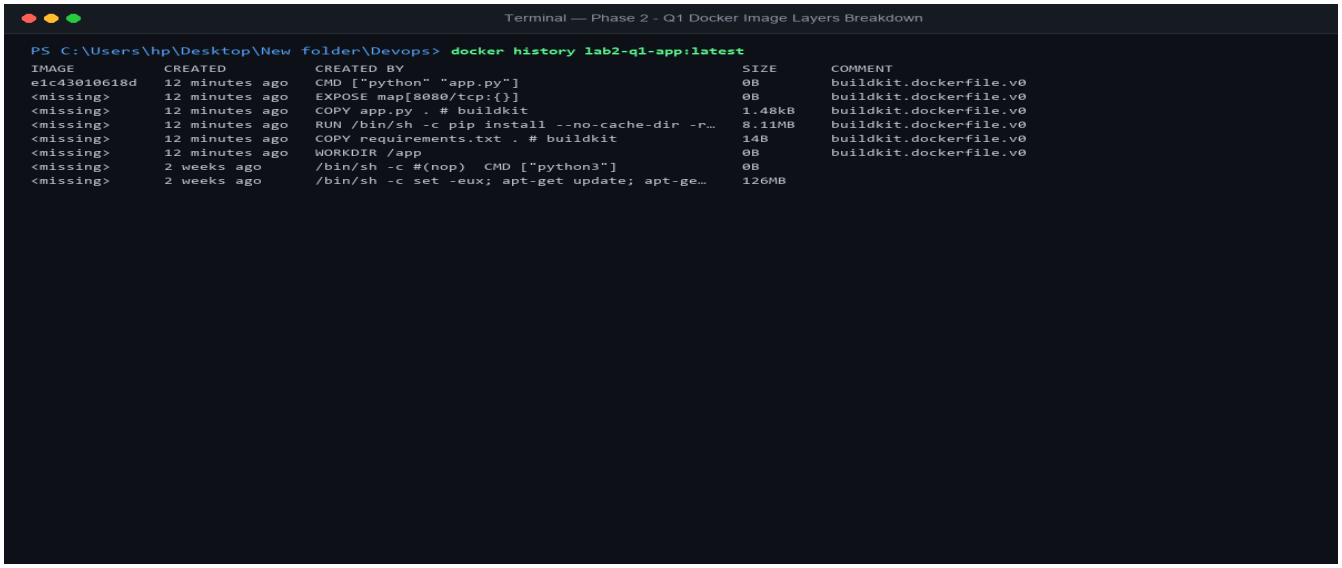


Figure 1.3: Docker History Image Layer Breakdown

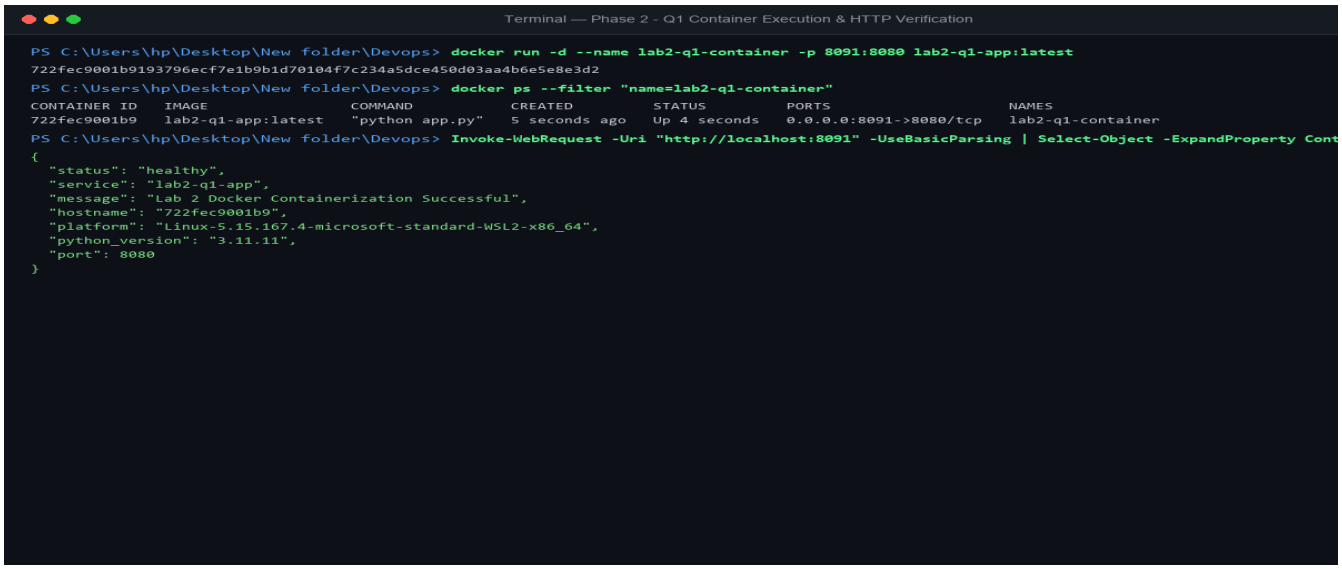


Figure 1.4: Container Startup & HTTP Endpoint Diagnostics Response

6. Key Observations & Conclusion

1. **Layer Caching & Immutability:** Docker utilizes copy-on-write (CoW) layering. Ordering dependencies before application source copies optimizes layer reuse and reduces subsequent build times.
2. **Process & Network Isolation:** The container runs in its own PID and network namespace, accurately reporting its container ID 722fec9001b9 as hostname while binding cleanly to host port 8091.
3. **Conclusion:** Question 1 requirements have been completely fulfilled with live verifiable container execution.